

image2gesture – A Classical Image Processing Pipeline for Real-Time Hand Gesture Recognition

James Whitfield

City College of New York

MS Computer Engineering

New York, New York

JAMES.WHITFIELD14@stu-mail.ccny.cuny.edu

Abstract

This paper presents image2gesture, a classical image processing pipeline for real-time hand gesture recognition using a standard laptop webcam. The system recognizes five gestures (open palm, closed fist, peace sign, thumbs up, and L) and applies a live image filter effect driven by the detected gesture. This pipeline utilizes HSV skin segmentation, morphological cleanup, convexity defect analysis for finger counting, and Hu moment shape descriptors. A hybrid classifier uses rule-based scoring as the primary decision and Hu nearest-neighbor matching as a tiebreaker for ambiguous gesture pairs. Evaluation is based on per-gesture accuracy against a test set, and various failure/pain points were identified and addressed. This 2D classical approach is then compared to sensor-fusion methods (i.e. Microsoft HoloLens 2).

1. Introduction

Gesture recognition has become a commonly studied alternative to physical input devices, as it allows hands-free control in contexts where keyboards, mice, or controllers are impractical. The ability to issue commands via hand gestures is particularly valuable in robotics and mixed reality, two industries that seem to be growing in perpetuity. This project is grounded in that context, where mixed reality headsets have native hand tracking that enable a myriad of responsive, gesture-driven controls. Image Processing is a field that attempts to enable humans in a parallel way through the implementation of computer vision and visual data processing. This project attempts to explore the natural question of how far a purely classical 2D pipeline can get toward the same goal as a mixed reality headset (without the depth sensors, neural networks, or other advanced hardware).

1.1. Scope

image2gesture targets five specific common hand gestures: open palm, closed fist, peace sign, thumbs up,

and an L. The system operates in real time via a standard webcam, using only classical image processing to recognize these gestures. HSV color segmentation, morphological filtering, contour analysis, and shape descriptors are utilized on each frame of the live webcam feed. The classified gesture is used to drive a live image filter effect, grounding the recognition in a tiny control system as a proof of concept.

This system was also designed to be an open-source resource for others to experiment with by way of a simple command line interface. To account for this, interpretability and tunability were prioritized.

1.2 Roadmap

Section 2 provides a basic relevant background of the aforementioned classical image processes. Section 3 will cover the full system design and pipeline architecture. Section 4 details the implementation of each stage of the system. Section 5 documents the development and tuning process. Section 6 presents evaluation results and qualitative failure analysis. Section 7 will compare the 2D classical approach and the sensor-fusion 3D methods used in MR. Section 8 concludes with identified limitations and potential improvements.

2. Background

Hand gesture recognition pipelines generally decompose into three main problems: isolating the hand from the background, extracting discriminative features from the isolated region, and mapping those features to a gesture label.

Color-based segmentation is the chosen approach for hand isolation (since the hand is relatively uniform in color). HSV was preferred over RGB to separate chromatic information from luminance.

Once the hand is isolated, gesture discrimination requires features that describe the shape rather than the color. Features are calculated and collected by building a contour around the hand and collecting aspect ratios, areas, and a finger count based on convex hulls and their defects. We also derive image moments (specifically Hu moments) that provide a compact, fixed-length descriptor to classify with.

These methods have several limitations, which are discussed further in Section 7.

3. System Design

image2gesture was designed around three goals: real-time operation on consumer hardware, interpretability of every pipeline decision, and tunability without massive changes to the source code. The architecture was shaped by these goals.

3.1. Design Goals and Technical Specifications

Each frame is processed through the pipeline independently and all stages target the quality, noise, and frame rate of a standard laptop webcam.

Fully implemented in Python, relying on OpenCV and NumPy.

Decision making is transparent and traceable through provided metrics (i.e. finger count, aspect ratio, confidence score, Hu moments) per frame. When the system misclassifies, the cause is visible and (ideally) tunable.

Tunability is handled through a central configuration file (config.py) that exposes all threshold parameters, kernel sizes, and classification ranges. The live HSV calibration mode (--calibrate) allows per-session tuning of the segmentation range without touching code.

CLI usage allows for simple injections of test images into the pipeline. Single images, directories, or a preset group of sample images for evaluation. Hu moment classification also relies on a set of sample images, which can be easily replaced/removed.

3.2. Hybrid Classification

The classifier combines two methods: a rule-based primary classifier and a Hu moment tiebreaker. Section 4 covers this in further detail, but the goal is to produce an interpretable score and winner for both method and evaluate a final best “guess”.

3.3.image2gesture Pipeline

The pipeline processes an image frame as described:

Frame (Original image frame from webcam)

|
V

Segmentation (Gaussian blur, HSV mask, ROI crop)

|
V

Morphology (opening - artifact/noise removal,
| closing - hole filling)

V

Contour Extraction (largest contour, convex hull,
| convexity defects, finger count, aspect ratio, Hu
| moments)

V

Classification (rule-based scoring, Hu tiebreaker)

|
V

Filter Output (live image filter effect based on gesture)

|
V

Display/Storage (debug view, filter view, screenshots,
evaluation)

This process repeats every frame for the live view. Each stage feeds the next with a single primary artifact: the segmentation stage produces a binary mask, morphology refines it, contour extraction pulls geometry and shape features from it, and classification maps those features to a gesture label. This linear structure keeps each stage independently testable and replaceable.

4. Implementation

4.1. Segmentation

Each frame is initially passed through a Gaussian blur to reduce high-frequency noise. The blurred frame is then converted to HSV and thresholded against a configured skin color range. The HSV bounds are configurable, but are designed to be overridden at runtime via a session config (hsv_config.json) generated via the --calibrate mode. This mode provides a live slider interface for adjusting the six HSV bounds against the webcam feed, with the resulting values saved to disk and loaded automatically on subsequent runs.

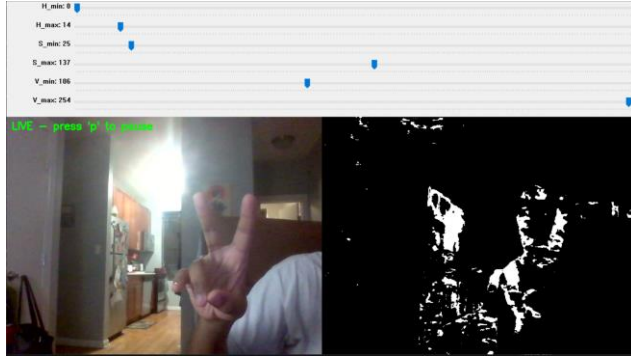


Figure 1: HSV tuner feed and sliders

A region on interest (ROI) mask is applied after thresholding, ignoring a configurable margin around each edge of the frame.

4.2. Morphology

The binary mask from segmentation is processed with two sequential morphological operations. First, an opening pass (erosion followed by dilation) with a small configurable kernel removes isolated pixel clusters too small to be a hand.

Second, a closing pass (dilation followed by erosion) with a larger configurable kernel fills internal gaps in the hand mask caused by uneven skin segmentation across the palm.

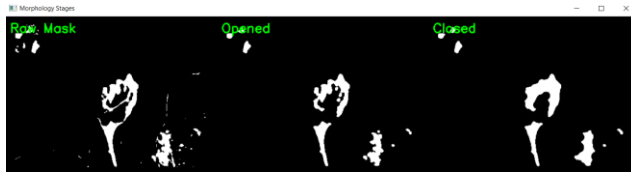


Figure 2: Morphology raw mask and passes

4.3. Contour Extraction

The largest valid contour is extracted from the morphological mask. A validity check (`is_valid_hand_contour()`) rejects contours that are too tall relative to their width, too thin, or too geometrically solid. Contours below a configurable minimum area threshold (`MIN_CONTOUR_AREA`) are also discarded as noise.

The accepted contour is smoothed with a small configurable value (`CONTOUR_SMOOTH_EPS`, the epsilon for use in OpenCV's `approxPolyDP` function) to reduce jagged edges before further analysis.

The convex hull is then computed, and convexity defects (the valleys between fingers) are extracted. Defects shallower than a dynamic depth threshold ($\text{contour area} \times \text{DEFECT_MIN_SCALE}$) are discarded as. This threshold is configurable. Finger count is estimated as the number of remaining defects plus one.

Three feature values are computed for classification: finger count from defect analysis, bounding box aspect ratio (width divided by height), and the seven Hu moments of the contour region.

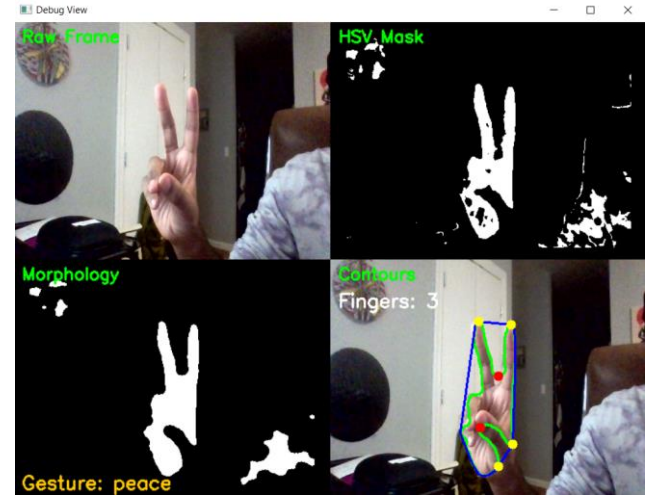


Figure 3: Three steps of pipeline, contour/feature output

4.4. Classification

Classification proceeds in two stages. The rule-based classifier scores each of the five gesture classes against the extracted finger count and aspect ratio, checking whether each value falls within the configured range for that gesture. Each satisfied rule contributes equally to a confidence score between 0.0 and 1.0. The gesture with the highest score wins.

For ambiguous gesture pairs where rule-based features overlap (thumb/L/peace), the Hu moment tiebreaker is invoked. The tiebreaker computes Euclidean distance between the frame's Hu feature vector and a stored mean template for each gesture in the pair, selecting the nearest.

Templates are precomputed from sample images per gesture via `--rebuild` and cached (`hu_templates.json`).

4.5. Filter Control

Each gesture label maps to a distinct image filter applied live to the raw camera frame:

Gesture	Filter
Open palm	Gaussian blur
Closed fist	Histogram equalization
Peace sign	Ordered dithering
Thumbs up	Negative
L	Negative

Figure 4: Filter control mapping

The filter is applied to the original unprocessed frame each cycle and rendered in a separate output window.

4.6. Display and Output

The system supports three display modes toggleable during live operation: debug view (d), which shows a 2×2 grid of the raw frame, HSV mask, morphological mask, and contour overlay; filter view (f), which shows the gesture-driven filtered output; and both views side by side (b). Frames can be saved to the output/ directory at any time with s. An evaluation mode (--eval) runs the full classification pipeline against labeled ground-truth folders and writes per-gesture accuracy, a confusion matrix, and a full classification report to output/eval_results.txt.

5. Evaluation

This section will describe the results of one run in --eval mode with preselected sample images and test images.

5.1. Methodology

[held-out per-gesture folders, controlled lighting + calibration]

5.2. Accuracy Table

[image or snippet of accuracy table]
[Per-gesture accuracy table]

5.3. Confusion Matrix

[Confusion matrix (highlight thumb/peace/L overlap)]

5.4. Qualitative Failure Analysis

[lighting sensitivity, arm bleed-in, fist weakness]

6. Discussion

6.1. Development & Tuning

Building the pipeline surfaced several non-obvious failure modes that required iterative diagnosis and targeted fixes. This section documents some of the most significant ones encountered.

Defect Depth Threshold

The initial implementation used a fixed minimum convexity defect depth to filter finger valleys from noise. This broke immediately across gestures as a threshold tuned for an open palm over-suppressed defects on a smaller or differently lit hand, while a threshold tuned for a compact gesture allowed noise defects to inflate finger counts. To account for this, the fixed value was replaced with a dynamic threshold scaled to contour area.

Hu L-Bias

Early evaluation revealed that the Hu moment classifier defaulted heavily toward the L gesture regardless of the input. Assumption here being that the L template happened to sit near the center of the feature space. This issue was sidestepped by mitigating its function architecturally to a tiebreaker role invoked only for ambiguous pairs. This evaluation cleared up a bit more with renewed sample data

Contour Self-Intersection

At higher epsilon values, OpenCV's approxPolyDP can produce self-intersecting contours. They'd cause problems in feature calculations anyway, but were causing crashes before the pipeline could get to that point. Source code was modified to allow frames to be skipped in this case, and the smoothening epsilon gained an upper limit.

Closed Fist

The closed fist was the most consistently problematic gesture throughout development. The HSV range was tuned against palm skin, but a closed fist presents primarily the dorsal surface of the hand, which is darker and less saturated. The resulting mask is fragmented and irregular in ways that morphological closing only partially recovers.

Confidence Threshold Evolution

The initial hybrid classifier used the Hu moments distance as a full fallback for any low-confidence rule-based result. This performed poorly due to the L-bias discussed previously. In this case, the rule-based classifier was performing well across all gestures on its own. The thresholds were tightened iteratively: the current logic invokes the Hu moments only when rule-based confidence falls below 0.75 on a known ambiguous pair and uses the Hu moments exclusively when confidence falls below 0.5. Above 0.75, the rule-based result is trusted directly. This narrowed the tiebreaker to the cases where it genuinely adds signal and kept it out of cases where it introduced noise.

6.2. Classical 2D vs. Sensor-Fusion

The failure modes documented above are not incidental. These stem from the limitations of a 2D pixel based image processing architecture.

HSV segmentation works when skin color is stable and distinguishable from the background. In practice, neither condition holds reliably. Illumination shifts the apparent

HSV values of the same skin under the same range. Clothing with similar hue bleeds into the mask. Skin tone variation across individuals means a range tuned for one person may perform poorly for another.

HoloLens 2, depth, and hand tracking

The HoloLens 2 hand tracking model infers full skeletal joint positions, including joints that are partially occluded. When a finger is hidden behind the palm, the model estimates its position from the visible joints and learned priors about hand kinematics. The image2gesture pipeline has no equivalent capability. This is a fundamental difference in the two approaches: 2D pixel appearance versus 3D geometry and joint kinematics.

The closed fist case illustrates the differences quite well. A 3D depth map can easily detect and understand a spherical volume at a known distance from the sensor. A 2D HSV mask sees a fragmented blob that happens to share a color range with the palm, poorly segmented and morphologically inconsistent.

In the robotics context that motivated this project, the trust requirements for gesture-driven commands are high. A misclassified gesture that triggers the wrong robot behavior carries real consequences. The image2gesture pipeline, as built, is not robust enough for that use case in uncontrolled environments. What it demonstrates is the ceiling of the 2D classical approach: reasonable accuracy under cooperative conditions, with fragility that is well-understood and traceable. A sensor-fusion system like the HoloLens 2 provides the robustness and confidence level that robot control requires.

7. Conclusion

The image2gesture pipeline demonstrates that a classical 2D image processing pipeline can achieve reliable hand gesture recognition under cooperative conditions using only a standard webcam. The five-gesture system accomplished its set goals by operating in real time, producing interpretable classification decisions at every stage, and being tunable without large source code changes. This is not without various limitations, all stemming from the inherent limits of 2D image processing in this context.

7.1. Improvements

Several concrete directions would meaningfully extend the pipeline within the 2D paradigm. YCbCr segmentation is more robust to illumination variation than HSV and would reduce the per-session calibration burden.

A dual-range HSV mask could be developed instead, to account for the palm/dorsal segmentation issue. Separated ranges for the two parts of the hand would improve the results of the early pipeline steps, increasing the final classifications.

Temporal smoothing across frames would reduce flickering in borderline cases without adding significant complexity, alongside a fully integrated GUI.

This project initially planned for the HoloLens 2's hand tracking output to act as a "ground truth". This could still be implemented with a custom HoloLens pipeline that uses the open source hl2ss package to evaluate features for the same 5 gestures for direct comparison.

7.2. Takeaways

Classical image processing is interpretable, portable, requires no training data beyond a small template bank, and fails in ways that are diagnosable and fixable. image2gesture is a working example of that. Every failure mode documented in this paper was traceable to a specific stage, and most were addressed through targeted parameter changes rather than architectural overhaul. This does not escape, however, the limits of the 2D pipeline. Due to this, classical image processing may not be ideal in fields like robotics, where confidence requirements are high.

References

- [1] [1] J. Whitfield, "image2gesture: A multi-stage classical image processing pipeline for real-time hand gesture recognition," GitHub, 2026. [Online]. Available: <https://github.com/whitfija/image2gesture>
- [2] [2] S. Mukherjee, "Shape Matching Using Hu Moments (C++/Python)," LearnOpenCV, 2019. [Online]. Available: <https://learnopencv.com/shape-matching-using-hu-moments-c-python/>
- [3] [3] M. Fernando and J. Wijayanayake, "Novel Approach to Use HU Moments with Image Processing Techniques for Real Time Sign Language Communication," arXiv preprint arXiv:2007.09859, Jul. 2020.
- [4] [4] A. Sharma, A. Mittal, S. Singh, and V. Awatramani, "Hand Gesture Recognition using Image Processing and Feature Extraction Techniques," Procedia Computer Science, vol. 173, pp. 181–190, 2020.
- [5] [5] D. Chakraborty, "OpenCV Contour Approximation (cv2.approxPolyDP)," PyImageSearch, Oct. 2021. [Online]. Available: <https://pyimagesearch.com/2021/10/06/opencv-contour-approximation/>
- [6] [6] J. C. Dibene and E. Dunn, "HoloLens 2 Sensor Streaming," arXiv preprint arXiv:2211.02648, Nov. 2022. [Online]. Available: <https://github.com/jdibenes/hl2ss>

Course Project Workload Analysis Table for Group Project

Tasks performed	James Whitfield	N/A
Research	100%	
Python Implementation	100%	
Report	100%	
Powerpoint	100%	
Overall	100%	



Team Member #1: James Whitfield

Signature

Team Member #2: N/A